Chapitre 2:

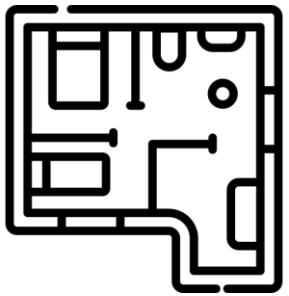
Introduction aux patrons de conception

Qu'est-ce qu'un patron de conception ?

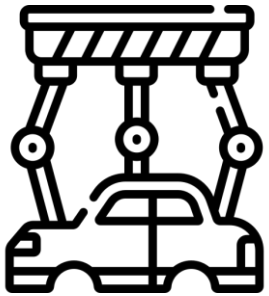
- Un problème déjà rencontré et déjà résolu.
- Une solution réutilisable à un problème qui revient régulièrement.
- Un modèle ou une conception servant de guide pour résoudre un problème.
- Une expérience capitalisée et applicable dans un nouveau contexte.
- Ce n’est pas une solution toute faite, mais une manière structurée de construire une solution.
- Plus formellement : « Une solution éprouvée à un problème courant dans un contexte donné. »

Qu'est-ce qu'un patron de conception ?

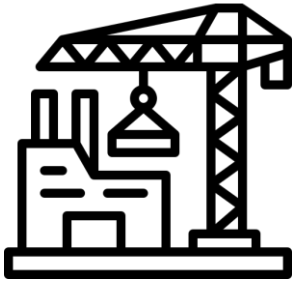
- Exemples concrets
  - Architecture: Plan d'une maison



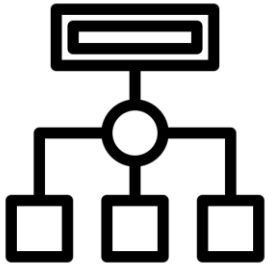
  - Industrie / fabrication: Méthode standard d'assemblage d'un produit



  - Construction: Technique standard pour résoudre un problème de construction



  - Informatique: Solution réutilisable à un problème fréquent de conception logicielle



Qu'est-ce qu'un patron de conception logicielle ?

- Un patron de conception logicielle est une solution réutilisable à un problème de conception récurrent.
- En génie logiciel, un patron de conception est une solution générale et réutilisable à un problème courant dans un contexte donné de conception logicielle.
- Il s'agit d'une description de la manière de résoudre un problème, utilisable dans de nombreuses situations différentes.
- L'objectif du processus de conception est de déterminer comment le code final sera structuré ou organisé en modules.
- Le résultat de ce processus est un modèle de solution abstrait, généralement exprimé à l'aide d'un langage de modélisation symbolique tel que l'UML.

Quand utiliser les patrons de conception

- Les patrons de conception sont particulièrement utiles lorsque :
  - Un même problème de conception revient régulièrement.
  - Le système devient complexe et nécessite une organisation claire des classes et des objets.
  - Le logiciel doit être facilement modifiable ou extensible.
  - On souhaite réutiliser une solution de conception éprouvée.
  - On veut faciliter la communication entre développeurs grâce à un vocabulaire commun.
  - Une solution simple devient difficile à maintenir ou risque de créer un fort couplage entre les composants.

Quand utiliser les patrons de conception

- Un patron de conception n'est pas une obligation: c'est un outil à utiliser lorsqu'il apporte une réelle valeur à la conception.
- Eviter les patrons de conception lorsque:
  - le problème est simple
  - une solution directe suffit
  - le patron ajoute une complexité inutile.

Avantages de l'utilisation des patrons de conception

- Accélérer le processus de développement en fournissant des paradigmes éprouvés et validés
- La réutilisation de patrons de conception aide à prévenir des problèmes subtils susceptibles d'entraîner des complications majeures, tout en améliorant la lisibilité du code pour les développeurs et les architectes qui les maîtrisent
- Les patrons de conception offrent des solutions générales, documentées sous une forme qui ne nécessite pas de détails liés à un problème spécifique

Avantages de l'utilisation des patrons de conception

- De plus, les modèles permettent aux développeurs de communiquer en utilisant des noms bien connus et bien compris pour désigner les interactions logicielles. Les modèles de conception courants peuvent être améliorés au fil du temps, ce qui les rend plus robustes que des conceptions ad hoc.
- Une solution standard à un problème de programmation courant permet une réutilisation logicielle à grande échelle.

Pourquoi étudier les patrons de conception?

- Ils offrent aux développeurs une boîte à outils pour résoudre des problèmes qui ont été résolus.

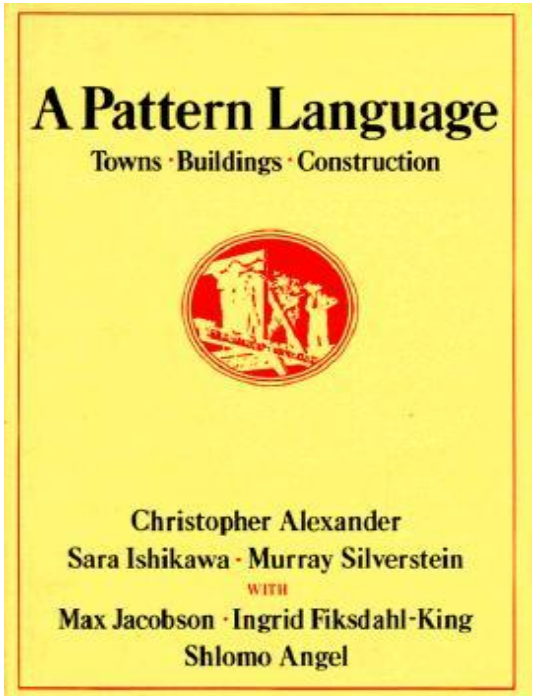
- Ils fournissent un vocabulaire commun aux développeurs.
- Le nom du patron lui-même contribue à établir ce vocabulaire.

- Ils aident à réfléchir à la manière de résoudre un problème logiciel.

Histoire des patrons de conception

- Les patrons ne sont pas nés dans le domaine du logiciel, ils sont apparus dans celui de l'urbanisme et de l'architecture.

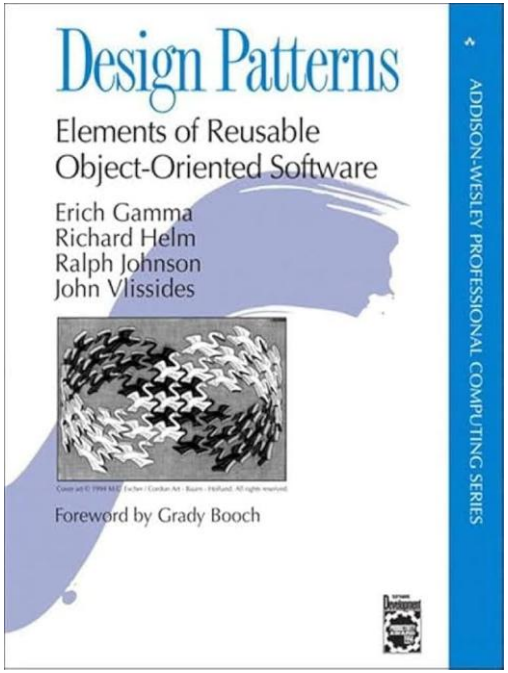
- Dans les années 1970, Christopher Alexander a développé un langage de patrons permettant au grand public de concevoir des bâtiments et des espaces publics plus harmonieux, fonctionnels et agréables à vivre.
  - Le premier ouvrage consacré aux « patterns », « A Pattern Language: Towns, Buildings, Construction », en documentant 253 patrons.
  - Chaque patron apportait une solution générale à un problème récurrent dans le domaine de l'urbanisme et de l'architecture.



Histoire des patrons de conception

- En 1987, Kent Beck et Ward Cunningham ont proposé de créer un langage de patrons pour la conception de logiciels.
  - Leur vision initiale était de permettre aux utilisateurs ordinaires d'écrire leurs propres programmes.

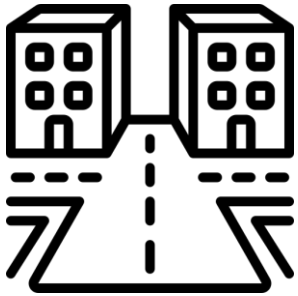
- Gang of Four (Bande des Quatre)
  - Le mouvement des patrons de conception logiciels a véritablement pris son essor après la publication de l'ouvrage ***Design Patterns: Elements of Reusable Object-Oriented Software*** [1994], proposant 23 patrons.
  - Les quatre auteurs: Erich Gamma, Richard Helm, Ralph Johnson et John Vlissides, sont collectivement connus sous le nom de **Gang of Four** ou **GoF**.



Les grandes étapes dans l'histoire des patrons de conception

Christopher Alexander a publié le premier livre sur les patrons : *A Pattern Language: Towns, Buildings, Construction*.

Il documente **253 patrons** dans les domaines de l'urbanisme et de l'architecture.



1977



1987

Kent Beck et Ward Cunningham ont proposé de créer un **langage de patrons pour la conception logicielle**, inspiré des travaux de Christopher Alexander sur la conception des villes et des bâtiments.

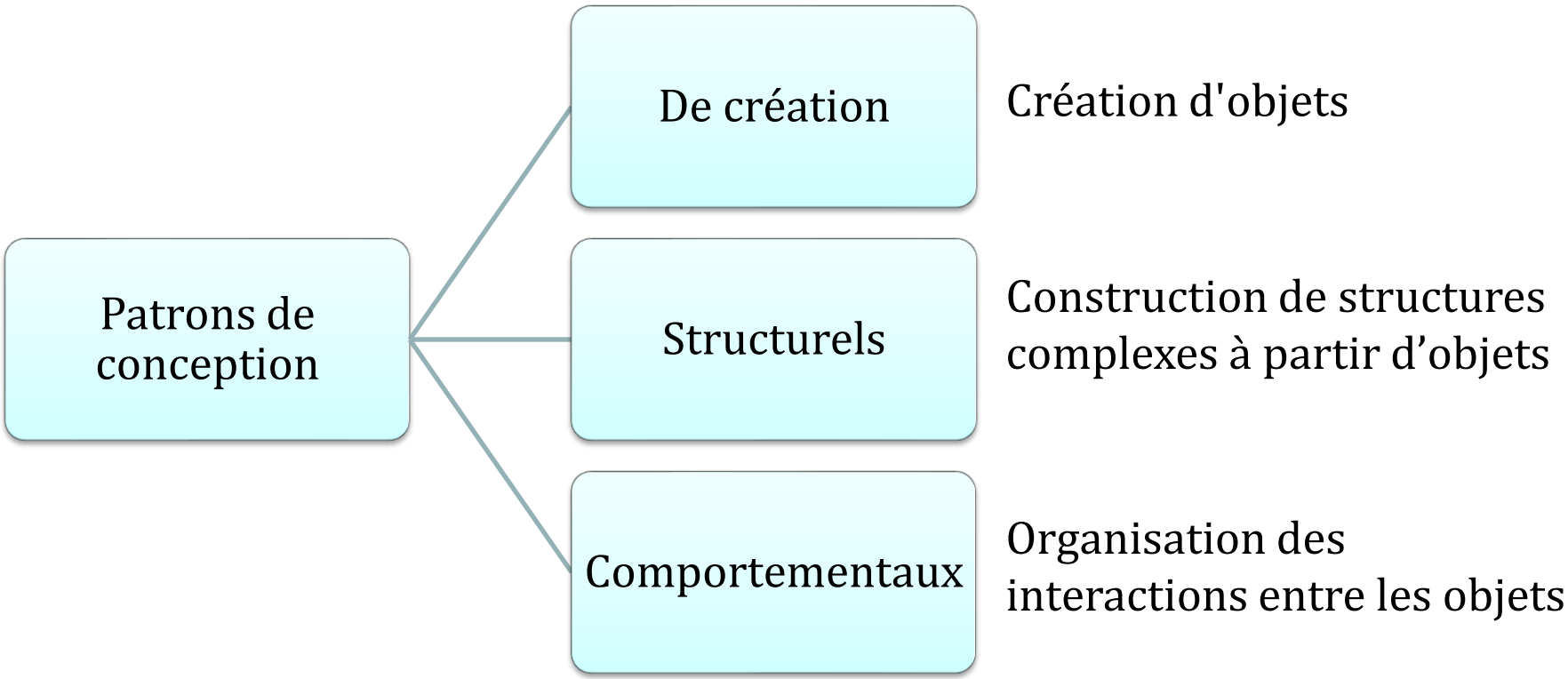
Le **« Gang of Four » (GoF)**, publient *Design Patterns: Elements of Reusable Object-Oriented Software*.

L'ouvrage présente **23 patrons de conception orientée objet**.

1994



Types des patrons de conception (GoF)



Le “Gang of Four”: Catalogue de patrons

- De création
  - Abstract Factory
  - Builder
  - Factory Method
  - Prototype
  - Singleton
- Structurels
  - Adapter
  - Bridge
  - Composite
  - Decorator
  - Façade
  - Flyweight
  - Proxy
- Comportementaux
  - Chain of Responsibility
  - Command
  - Interpreter
  - Iterator
  - Mediator
  - Memento
  - Observer
  - State
  - Strategy
  - Template Method
  - Visitor

Modèle de présentation des patrons

- Intention
  - Brève description du patron et de son objectif
- Alias
  - Autres noms sous lesquels ce patron est connu
- Motivation
  - Scénario illustrant l'utilisation du patron
- Applicabilité
  - Circonstances dans lesquelles le patron s'applique
- Structure
  - Représentation graphique du patron utilisant une notation UML modifiée
- Participants
  - Classes et/ou objets participants et leurs responsabilités

Modèle de présentation des patrons

- Collaborations
  - la manière dont les participants coopèrent pour s'acquitter de leurs responsabilités
- Conséquences
  - les résultats de la mise en œuvre, les avantages et les contraintes
- Mise en œuvre
  - les pièges, conseils et techniques, ainsi que les aspects liés au langage
- Exemples de code
  - exemples d'implémentation en C++, Java, C#, Python, Smalltalk, etc.
- Utilisations connues
  - exemples tirés de systèmes existants
- Patrons associés
  - discussion sur d'autres patrons liés à celui-ci